

Week 7: Regular Expressions and Web Scraping

PM 566: Introduction to Health Data Science

George G. Vega Yon and Kelly Street

Today's goals

- Introduction to Regular Expressions
- Understand the fundamentals of Web Scraping
- Learn how to use an API

Regular Expressions: What is it?

A regular expression (shortened as regex or regexp; also referred to as rational expression) is a sequence of characters that define a search pattern. – [Wikipedia](#)



Regular Expressions: Why should you care?

We can use Regular Expressions for:

- Validating data fields, email address, numbers, etc.
- Searching text in various formats, e.g., addresses, there are many ways to write an address.
- Replace text, e.g., different spellings, *Storm*, *Stôrm*, *Stórm* to *Storm*.
- Remove text, e.g., tags from an HTML text, *<name>George</name>* to *George*.

Regex 101: Metacharacters

What makes *regex* special is metacharacters. While we can always use *regex* to match literals like `dog`, `human`, `1999`, we only make use of all *regex* power when using metacharacters:

- `.` Any character except new line
- `^` beginning of the text
- `$` end of the text
- `[expression]` Match any single character in “expression”, e.g.
 - `[0123456789]` Any digit
 - `[0-9]` Any digit in the range 0-9
 - `[a-z]` Lower-case letters
 - `[A-Z]` Upper-case letters
 - `[a-zA-Z]` Lower or upper case letters.
 - `[a-zA-Z0-9]` Any alpha-numeric

Regex 101: Metacharacters (cont. 1)

- `[^regex]` Match any except those in `regex`, e.g.
 - `[^0123456789]` Match any except a number
 - `[^0-9]` Match anything except in the range 0-9
 - `[^./ ]` any except dot, slash, and space.

Regex 101: Metacharacters (cont. 2)

Ranges, e.g., `0–9` or `a–z`, are locale- and implementation-dependent, meaning that the range of lower case letters may vary depending on the OS's language. To solve for this problem, you could use [Character classes](#). Some examples:

- `[[:lower:]]` lower case letters in the current locale, could be `[a–z]`
- `[[:upper:]]` upper case letters in the current locale, could be `[A–Z]`
- `[[:alpha:]]` upper and lower case letters in the current locale, could be `[a–zA–Z]`
- `[[:digit:]]` Digits: 0 1 2 3 4 5 6 7 8 9
- `[[:alnum:]]` Alpha numeric characters `[[:alpha:]]` and `[[:digit:]]`.
- `[[:punct:]]` Punctuation characters: ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { | } ~.

For example, in the locale `en_US`, the word `Hóla` IS NOT fully matched by `[a–zA–Z]+`, but IT IS fully matched by `[[:alpha:]]+`.

Regex 101: Metacharacters (cont. 3)

Other important metacharacters:

- `\s` white space, equivalent to `[\r\n\t\f\v ]`
- `|` or (logical or).

Regex 101: Metacharacters (cont. 4)

These usually come together with specifying how many times (repetition):

- `regex?` Zero or one match.
- `regex*` Zero or more matches
- `regex+` One or more matches
- `regex{n, }` At least `n` matches
- `regex{, m}` at most `m` matches
- `regex{n, m}` Between `n` and `m` matches.

Where `regex` is a regular expression

Regex 101: Metacharacters (cont. 5)

There are other operators that can be very useful,

- `( regex )` Group capture.
- `( ?: regex )` Group operation without capture.
- `A ( ?=B )` Look ahead (match). Find expression A where expression B follows.
- `A ( ?!B )` Look ahead (don't match). Find expression A where expression B does not follow.
- `( ?<=B ) A` Look behind (match). Find expression A where expression B precedes.
- `( ?<!B ) A` Look behind (don't match). Find expression A where expression B does not precede.

Group captures can be reused with `\\1`, `\\2`, ..., `\\n`, referring to the first group, second group, etc.

More (great) information here <https://regex101.com/>

Regex 101: Examples

1. `.{5}` Match **any character** (except line end) five times.
2. `n{2}` Match the letter **n** twice.
3. `[0-9]` Match **any number** once.
4. `[0-9]+` Match **any number** at least once.
5. `[a-zA-Z]+` Match **any lower or upper case letter** at least once.
6. `\\s[a-zA-Z]+\\s` Match a **space**, **any lower or upper case letter** at least once, and a **space**.
7. `\\s[[:alpha:]]+\\s` Same as before but this time with the `[[:alpha:]]` character class.

Regex 101: Examples (cont. 1)

Here we are extracting the **first occurrence** of the following regular expressions (using `stringr::str_extract()`):

regex	Hanna Perez [name]	The year was 1999	HaHa, @abc said that	GoGo trojans #2025!
.{5}	Hanna	The	HaHa,	GoGo
n{2}	nn			
[0-9]		1		2
[0-9]+		1999		2025
[a-zA-Z]+	Hanna	The	HaHa	GoGo
\\s[a-zA-Z]+\\s	Perez	year	said	trojans
\\s[:,alpha:]]+\\s	Perez		said	trojans

Regex 101: Examples (cont. 3)

8. `[a-zA-Z]+ [a-zA-Z]+` Match two sets of letters separated by one space.
9. `[a-zA-Z]+\s?` Match one set of letters, maybe followed by a white space.
10. `([a-zA-Z]+)\s1` Match **any lower or upper case letter** at least once and then match the same pattern again.
11. `(@|#)[a-z0-9]+` Match either the `@` or `#` symbol, followed by one or more **lower case letter or number**.
12. `(?<=#|@)[a-z0-9]+` Match one or more **lower case letter or number** that follows either the `@` or `#` symbol.
13. `\[ [a-z]+\]` Match the symbol `[`, at least one **lower case letter**, then symbol `]`.

Regex 101: Examples (cont. 2)

regex	Hanna Perez [name]	The year was 1999	HaHa, @abc said that	GoGo trojans #2025!
[a-zA-Z]+ [a-zA-Z]+	Hanna Perez	year was	abc said	GoGo trojans
[a-zA-Z]+\s?	Hanna	The	HaHa	GoGo
([a-zA-Z]+\s)\s1	nn		HaHa	GoGo
(@ #)[a-z0-9]+			@abc	#2025
(?<=# @)[a-z0-9]+			abc	2025

Regex 101: Examples (cont. 3)

regex	Hanna Perez	The ☐ year	HaHa, @abc	GoGo trojans
	[name]	was 1999	said that	#2025!
\\[a-z]+\\	[name]			

Regex 101: Functions in R

1. Find in text: `grepl()`, `stringr::str_detect()`.
2. Similar to `which()`, which elements are TRUE `grep()`, `stringr::str_which()`
3. Replace the first instance: `sub()`, `stringr::str_replace()`
4. Replace all instances: `gsub()`, `stringr::str_replace_all()`
5. Extract text: `regmatches()` plus `regexpr()`, `stringr::str_extract()` and `stringr::str_extract_all()`.



Regex 101: Functions in R (cont. 1)

For example, like in Twitter, let's create a regex that matches usernames or hashtags with the following pattern:

```
(@|#)([[:alnum:]]+)
```

Code	@Hanna Perez [name] #html	The @ year was 1999	HaHa, @abc said that @z
<code>str_detect(text, pattern)</code> or <code>grepl(pattern, text)</code>	TRUE	TRUE	TRUE
<code>str_extract(text, pattern)</code> or <code>regmatches(...)</code>	@Hanna	@	@abc
<code>str_extract_all(text, pattern)</code>	@Hanna, #html	@	@abc, @z

Regex 101: Functions in R (cont. 2)

Pattern: `(@|#)([[:alnum:]]+)`

Code	@Hanna Perez [name] #html	The @ year was 1999	HaHa, @abc said that @z
<code>str_replace(text, pattern, "\\1justinbieber")</code> or <code>sub(...)</code>	@justinbieber Perez [name] #html	The @justinbieber year was 1999	HaHa, @justinbieber said that @z
<code>str_replace_all(text, pattern, "\\1justinbieber")</code> or <code>gsub(...)</code>	@justinbieber Perez [name] #justinbieber	The @justinbieber year was 1999	HaHa, @justinbieber said that @justinbieber

Note: It renders oddly in the table, but there is no space or line break in the group replacement, `\\1`.

Data

This week we will continue using our text mining dataset:

```
1 # Where are we getting the data from
2 mts_url <- "https://github.com/USCbiostats/data-science-data/raw/main/mtsamples.csv"
3
4 # Downloading the data to a tempfile (so it is destroyed afterwards)
5 tmp <- tempfile(pattern = "mtsamples", fileext = ".csv")
6
7 # We should be downloading this, ONLY IF this was not downloaded already
8 # otherwise is just a waste of time.
9 if (!file.exists(tmp)) {
10   download.file(
11     url      = mts_url,
12     destfile = tmp,
13     # method  = "libcurl", timeout = 1000 (you may need this option for some URLs)
14   )
15 }
16
17 # read the file
18 mtsamples <- read.csv(tmp, header = TRUE, row.names = 1)
```

Regex Lookup Text: Tumor

For each entry, we want to know if it is tumor-related. For that we can use the following code:

```
1 # How many entries contain the word tumor?
2 sum(grepl("tumor", mtsamples$description, ignore.case = TRUE))
```

```
[1] 80
```

```
1 # Generating a column tagging tumor
2 mtsamples$tumor_related <- grepl("tumor", mtsamples$description, ignore.case = TRUE)
3
4 # Taking a look at a few examples
5 mtsamples$description[mtsamples$tumor_related == TRUE][1:3]
```

```
[1] " Transurethral resection of a medium bladder tumor (TURBT), left lateral wall."
```

```
[2] " Transurethral resection of the bladder tumor (TURBT), large."
```

```
[3] " Cystoscopy, transurethral resection of medium bladder tumor (4.0 cm in diameter), and direct bladder biopsy."
```

Notice the `ignore.case = TRUE`. This is equivalent to transforming the text to lower case using `tolower()` before passing the text to the regular expression function.

Regex Lookup text: Pronoun of the patient

Now, let's try to guess the pronoun of the patient. To do so, we could tag by using the words *he*, *his*, *him*, *they*, *them*, *theirs*, *ze*, *hir*, *hirs*, *she*, *hers*, *her* (see [this article on sexist text](#)):

```
1 mtsamples$pronoun <- str_extract(  
2   string = tolower(mtsamples$transcription),  
3   pattern = "he|his|him|they|them|theirs|ze|hir|hirs|she|hers|her"  
4 )
```

What is the problem with this approach?

Regex Lookup text: Pronoun of the patient (cont. 1)

```
1 mtsamples$transcription[1]
```

```
[1] "SUBJECTIVE:, This 23-year-old white female presents with complaint of allergies. She used to have allergies when she lived in Seattle but she thinks they are worse here. In the past, she has tried Claritin, and Zyrtec. Both worked for short time but then seemed to lose effectiveness. She has used Allegra also. She used that last summer and she began using it again two weeks ago. It does not appear to be working very well. She has used over-the-counter sprays but no prescription nasal sprays. She does have asthma but doest not require daily medication for this and does not think it is flaring up.,MEDICATIONS: , Her only medication currently is Ortho Tri-Cyclen and the Allegra.,ALLERGIES: , She has no known medicine allergies.,OBJECTIVE:,Vitals: Weight was 130 pounds and blood pressure 124/78.,HEENT: Her throat was mildly erythematous without exudate. Nasal mucosa was erythematous and swollen. Only clear drainage was seen. TMs were clear.,Neck: Supple without adenopathy.,Lungs: Clear.,ASSESSMENT:, Allergic rhinitis.,PLAN:,1. She will try Zyrtec instead of Allegra again. Another option will be to use loratadine. She does not think she has prescription coverage so that might be cheaper.,2. Samples of Nasonex two sprays in each nostril given for three weeks. A prescription was written as well."
```

```
1 mtsamples$pronoun[1]
```

```
[1] "his"
```

Regex Lookup text: Pronoun of the patient (cont. 1)

To correct this issue, we can make our regular expression more precise:

```
(?<=\W|^)
```

```
(he|his|him|they|them|theirs|ze|hir|hirs|she|hers|her)(?=\W|$)
```

Bit by bit this is:

- `(?<=regex)` look behind for...
 - `\W` any non-alphanumeric character, this is equivalent to `[^[:alnum:]]`, `|` or
 - `^` the beginning of text
- `he|his|him...` any of these words
- `(?=regex)` followed by...
 - `\W` any non-alphanumeric character, this is equivalent to `[^[:alnum:]]`, `|` or
 - `$` the end of the text.

Regex Lookup text: Pronoun of the patient (cont. 2)

Let's use this new pattern:

```
1 mtsamples$pronoun <- str_extract(  
2   string = tolower(mtsamples$transcription),  
3   pattern = "(?<=\\W|^)(he|his|him|they|them|theirs|ze|hir|hirs|she|hers|her)(?=\\W|$)"  
4 )  
5 mtsamples$pronoun[1:10]
```

```
[1] "she" "he"  "he"  NA     NA     "she" "she" NA     NA     NA
```


Regex Lookup text: Pronoun of the patient (cont. 3)

```
1 table(mtsamples$pronoun, useNA = "always")
```

he	her	him	his	she	them	they	<NA>
1073	554	39	524	1261	23	96	1429

Regex Extract Text: Type of Cancer

- Imagine now that you need to see the types of cancer mentioned in the data.
- For simplicity, let's assume that, if specified, it is in the form of **TYPE cancer**, i.e. single word.
- We are interested in the word before cancer, how can we capture this?

Regex Extract Text: Type of Cancer (cont 1.)

We can just try to **extract** the phrase "[some word] cancer", in particular, we could use the following regular expression:

```
[[:alnum:]]-_{4,}\s*cancer
```

Where

- `[[:alnum:]]-_{4,}` captures any alphanumeric character, including `-` and `_`. Furthermore, for this match to work there must be at least 4 characters,
- `\s*` captures 0 or more white-spaces, and
- `cancer` captures the word cancer

Regex Extract Text: Type of Cancer (cont. 2)

```
1 mtsamples$cancer_type <- str_extract(tolower(mtsamples$keywords), "[[:alnum:]-_]{4,}\\s*cancer")
2 table(mtsamples$cancer_type)
```

anal cancer	bladder cancer	breast cancer	colon cancer
1	8	21	14
endometrial cancer	esophageal cancer	lung cancer	ovarian cancer
5	2	13	1
papillary cancer	prostate cancer	uterine cancer	
3	17	7	

Web Scraping

Fundamentals of Web Scraping

What?

Web scraping, web harvesting, or web data extraction is data scraping used for extracting data from websites – [Wikipedia](#)

How?

- The [rvest](#) R package provides various tools for reading and processing web data.
- Under the hood, [rvest](#) is a wrapper of the [xml2](#) and [httr](#) R packages.

(in the case of [dynamic websites](#), take a look at [selenium](#))

Web scraping raw HTML: Example

We would like to capture the table of [COVID-19 death rates](https://en.wikipedia.org/wiki/COVID-19_pandemic_death_rates_by_country) per country directly from Wikipedia.

```
1 library(rvest)
2 library(xml2)
3
4 # Reading the HTML table with the function xml2::read_html
5 covid <- read_html(
6   x = "https://en.wikipedia.org/wiki/COVID-19_pandemic_death_rates_by_country"
7 )
8
9 # Let's look at the the output
10 covid
```

```
{html_document}
```

```
<html class="client-nojs vector-feature-language-in-header-enabled vector-feature-language-in-main-page-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector-sticky-header-enabled vector-toc-available" lang="en" dir="ltr">
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
[2] <body class="skin--responsive skin-vector skin-vector-search-vue mediawik ...
```


Web scraping raw HTML: Example (cont. 1)

- We want to get the HTML table that shows up in the doc. To do so, we can use the function `xml2::xml_find_all()` and `rvest::html_table()`
- The first will locate the place in the document that matches a given **XPath** expression.
- **XPath**, XML Path Language, is a query language to select nodes in a XML document.
- A nice tutorial can be found [here](#)
- Modern Web browsers make it easy to use XPath!

Live Example! (inspect elements in [Google Chrome](#), [Mozilla Firefox](#), [Internet Explorer](#), and [Safari](#))

Web scraping with **xml2** and the **rvest** package (cont. 2)

Now that we know the path, let's use that and extract the table:

```
1 table <- xml_find_all(covid, xpath = '/html/body/div[2]/div/div[3]/  
2 print(table) # collection of tables in XML
```

```
{xml_nodeset (1)}
```

```
[1] <table class="wikitable sortable sort-under static-row-numbers mw-datatab
```

```
...
```

Web scraping with `xml2` and the `rvest` package (cont. 3)

We're getting closer!

```
1 table <- html_table(table)
2 print(table) # list of tibbles
```

```
[[1]]
```

```
# A tibble: 240 × 4
```

	Country <chr>	`Deaths / million` <chr>	Deaths <chr>	Cases <chr>
1	World[a]	892	7,101,682	778,602,438
2	Peru	6,603	221,060	4,532,724
3	Bulgaria	5,679	38,767	1,339,176
4	North Macedonia	5,429	9,991	352,093
5	Bosnia and Herzegovina	5,119	16,406	404,289
6	Hungary	5,072	49,124	2,238,461
7	Croatia	4,807	18,784	1,360,993
8	Slovenia	4,686	9,914	1,363,849
9	Georgia	4,519	17,151	1,864,386
10	Montenegro	4,317	2,654	251,280

```
# i 230 more rows
```

Web scraping with `xml2` and the `rvest` package (cont. 4)

There it is!

```
1 table <- table[[1]]  
2 print(table) # hooray!
```

A tibble: 240 × 4

	Country <chr>	`Deaths / million` <chr>	Deaths <chr>	Cases <chr>
1	World[a]	892	7,101,682	778,602,438
2	Peru	6,603	221,060	4,532,724
3	Bulgaria	5,679	38,767	1,339,176
4	North Macedonia	5,429	9,991	352,093
5	Bosnia and Herzegovina	5,119	16,406	404,289
6	Hungary	5,072	49,124	2,238,461
7	Croatia	4,807	18,784	1,360,993
8	Slovenia	4,686	9,914	1,363,849
9	Georgia	4,519	17,151	1,864,386
10	Montenegro	4,317	2,654	251,280

i 230 more rows

* Web APIs

What?

A Web API is an application programming interface for either a web server or a web browser. – [Wikipedia](#)

Some examples include: [twitter API](#), [facebook API](#), [Gene Ontology API](#)

How?

You can request data, the **GET method**, post data, the **POST method**, and do many other things using the [HTTP protocol](#).

How in R?

For this part, we will be using the [httr\(\)](#) package, which is a wrapper of the [curl\(\)](#) package, which in turn provides access to the [curl](#) library that is used to communicate with APIs.

* Web APIs with curl

A diagram illustrating the structure of a URL. The URL is `http://www.domain.com:1234/path/to/resource?a=b&x=y`. Red horizontal lines segment the URL into five parts, each with a label below it: `http` is labeled 'protocol'; `://www.domain.com` is labeled 'host'; `:1234` is labeled 'port'; `/path/to/resource` is labeled 'resource path'; and `?a=b&x=y` is labeled 'query'.

`http://www.domain.com:1234/path/to/resource?a=b&x=y`

protocol host port resource path query

Structure of a URL (source: [“HTTP: The Protocol Every Web Developer Must Know - Part 1”](#))

* Web APIs with curl

Under the hood, the `httr` (and thus `curl`) sends request somewhat like this

```
1 curl -X GET https://google.com -w "%{content_type}\n%{http_code}\n"
```

A get request (`-X GET`) to `https://google.com`, which also includes (`-w`) the following: `content_type` and `http_code`:

```
1 <HTML><HEAD><meta http-equiv="content-type" content="text/html;char
2 <TITLE>301 Moved</TITLE></HEAD><BODY>
3 <H1>301 Moved</H1>
4 The document has moved
5 <A HREF="https://www.google.com/">here</A>.
6 </BODY></HTML>
7 text/html; charset=UTF-8
8 301
```

We use the `httr` R package to make life easier.

* Web API Example 1: Gene Ontology

- We will make use of the [Gene Ontology API](#).
- We want to know what genes (human or not) are **involved** in the function **antiviral innate immune response** (go term [GO:0140374](#)), looking only at those annotations that have evidence code [ECO:0000006](#) (experimental evidence):

```
1 library(httr)
2 go_query <- GET(
3   url    = "http://api.geneontology.org/",
4   path   = "api/bioentity/function/GO:0140374/genes",
5   query  = list(
6     evidence          = "ECO:0000006",
7     relationship_type = "involved_in"
8   ),
9   # May need to pass this option to curl to allow to wait for at least
10  # 60 seconds before returning error.
11  config = config(
12    connecttimeout = 60
13  )
14 )
```

We could have also passed the full URL directly...

* Web API Example 1: Gene Ontology (cont. 1)

Let's take a look at the curl call:

```
1 curl -X GET "http://api.geneontology.org/api/bioentity/function/GO:
```

What `httr::GET()` does:

```
1 > go_query$request
2 ## <request>
3 ## GET http://api.geneontology.org/api/bioentity/function/GO:014037
4 ## Output: write_memory
5 ## Options:
6 ## * useragent: libcurl/7.58.0 r-curl/4.3 httr/1.4.1
7 ## * connecttimeout: 60
8 ## * httpget: TRUE
9 ## Headers:
10 ## * Accept: application/json, text/xml, application/xml, */*
```

* Web API Example 1: Gene Ontology (cont. 2)

Let's take a look at the response:

```
1 go_query
```

```
Response [https://api.geneontology.org/api/bioentity/function/GO:0140374/genes?
evidence=ECO%3A00000006&relationship_type=involved_in]
  Date: 2025-08-28 23:29
  Status: 200
  Content-Type: application/json
  Size: 116 kB
```

Remember the codes:

- 1xx: Information message
- 2xx: Success
- 3xx: Redirection
- 4xx: Client error
- 5xx: Server error

* Web API Example 1: Gene Ontology (cont. 3)

We can extract the results using the `http::content()` function:

```
1 dat <- content(go_query)
2 dat <- lapply(dat$associations, function(a) {
3   data.frame(
4     Gene      = a$subject$id,
5     taxon_id  = a$subject$taxon$id,
6     taxon_label = a$subject$taxon$label
7   )
8 })
9 dat <- do.call(rbind, dat)
10 str(dat)
```

```
'data.frame':  100 obs. of  3 variables:
 $ Gene      : chr  "UniProtKB:H9GKP0" "UniProtKB:H9GSI3" "UniProtKB:A0A287AMJ0" "UniProtKB:A0A287AKR1" ...
 $ taxon_id  : chr  "NCBITaxon:28377" "NCBITaxon:28377" "NCBITaxon:9823" "NCBITaxon:9823" ...
 $ taxon_label: chr  "Anolis carolinensis" "Anolis carolinensis" "Sus scrofa" "Sus scrofa" ...
```

* Web API Example 1: Gene Ontology (cont. 4)

The structure of the result will depend on the API. In this case, the output was a JSON file, so the content function returns a list in R. In other scenarios it could return an XML object.

```
1 knitr::kable(head(dat),  
2   caption = "Genes experimentally annotated with the function\  
3   **antiviral innate immune response** (GO:0140374)"  
4   )
```

Genes experimentally annotated with the function **antiviral innate immune response (GO:0140374)**

Gene	taxon_id	taxon_label
UniProtKB:H9GKP0	NCBITaxon:28377	Anolis carolinensis
UniProtKB:H9GSI3	NCBITaxon:28377	Anolis carolinensis
UniProtKB:A0A287AMJ0	NCBITaxon:9823	Sus scrofa
UniProtKB:A0A287AKR1	NCBITaxon:9823	Sus scrofa
UniProtKB:F7EI59	NCBITaxon:9544	Macaca mulatta

* Web API Example 2: Using Tokens

- Sometimes, APIs are not completely open, you need to register.
- The API may require to login (user+password), or pass a token.
- In this example, I'm using a token which I obtained [here](#)
- You can find information about the [National Centers for Environmental Information](#) API [here](#)

* Web API Example 2: Using Tokens (cont. 1)

- The way to pass the token will depend on the API service.
- Some require authentication, others need you to pass it as an argument of the query, i.e., directly in the URL.
- In this case, we pass it on the header.

This is equivalent to using the following query

```
1 curl --header "token: [YOUR TOKEN HERE]" \  
2 https://www.ncdc.noaa.gov/cdo-web/api/v2/stations?limit=1000
```

Note: This won't run, you need to get your own token

* Web API Example 2: Using Tokens (cont. 2)

Again, we can recover the data using the `content()` function:

```
1 ans <- content(stations_api)
2 ans$results[[1]]
3 ## $elevation
4 ## [1] 139
5 ##
6 ## $mindate
7 ## [1] "1948-01-01"
8 ##
9 ## $maxdate
10 ## [1] "2014-01-01"
11 ##
12 ## $latitude
13 ## [1] 31.5702
14 ##
15 ## $name
16 ## [1] "ABBEVILLE, AL US"
17 ##
18 ## $datacoverage
19 ## [1] 0.8813
20 ##
21 ## $id
22 ## [1] "COOP:010008"
```

* Web API Example 3: HHS health recommendation

Here is a last example. We will use the Department of Health and Human Services API for “[...] demographic-specific health recommendations” (details at health.gov)

```
1 health_advises <- GET(  
2   url   = "https://health.gov/",  
3   path  = "myhealthfinder/api/v3/myhealthfinder.json",  
4   query = list(  
5     lang = "en",  
6     age  = "32",  
7     sex  = "male",  
8     tobaccoUse = 0  
9   ),  
10  config = c(  
11    add_headers(accept = "application/json"),  
12    config(connecttimeout = 60)  
13  )  
14 )
```


* Web API Example 3: HHS health recommendation (cont. 1)

Let's see the response

```
1 health_advises
```

```
Response [https://odphp.health.gov/myhealthfinder/api/v3/myhealthfinder.json?
lang=en&age=32&sex=male&tobaccoUse=0]
```

```
Date: 2025-08-28 23:29
```

```
Status: 200
```

```
Content-Type: application/json
```

```
Size: 310 kB
```

```
{
  "Result": {
    "Error": "MyHealthfinder API version 3 is no longer supported. Please...",
    "Total": 18,
    "Query": {
      "ApiVersion": "3",
      "ApiType": "myhealthfinder",
      "TopicId": null,
      "CategoryId": null,
      "Keyword": null,
      ...
    }
  }
}
```

* Web API Example 3: HHS health recommendation (cont. 2)

```
1 # Extracting the content
2 health_advises_ans <- content(health_advises)
3
4 # Getting the titles
5 txt <- with(health_advises_ans$Result$Resources, c(
6   sapply(all$Resource, "[", "Title"),
7   sapply(some$Resource, "[", "Title"),
8   sapply(`You may also be interested in these health topics:`$Resource, "[", "Title")
9 ))
10 cat(txt, sep = "; ")
```

Quit Smoking; Hepatitis C Screening: Questions for the Doctor; Protect Yourself from Seasonal Flu; Talk with Your Doctor About Depression; Get Your Blood Pressure Checked; Get Tested for HIV; Get Vaccines to Protect Your Health (Adults Ages 19 to 49 Years); Drink Alcohol Only in Moderation; Talk with Your Doctor About Drug Misuse and Substance Use Disorder; Aim for a Healthy Weight; Testing for Syphilis: Questions for the Doctor; Eat Healthy; Protect Yourself from Hepatitis B; Testing for Latent Tuberculosis: Questions for the Doctor; Manage Stress; Alcohol Use: Conversation Starters; Get Active; Quitting Smoking: Conversation Starters

Summary

- We learned about regular expressions with the package **stringr** (a wrapper of **stringi**)
- We can use regular expressions to detect (`str_detect()`), replace (`str_replace()`), and extract (`str_extract()`) expressions.
- We looked at web scraping using the **rvest** package (a wrapper of **xml2**).
- We extracted elements from the HTML/XML using `xml_find_all()` with XPath expressions.
- We also used the `html_table()` function from **rvest** to extract tables from HTML documents.
- We took a quick review on Web APIs and the Hyper-text-transfer-protocol (HTTP).
- We used the **httr** R package (wrapper of **curl**) to make **GET** requests to various APIs
- We even showed an example using a token passed via the **header**.
- Once we got the responses, we used the `content()` function to extract the message of the response.

Detour on CURL options

Sometimes you will need to change the default set of options in CURL. You can checkout the list of options in `curl::curl_options()`. A common hack is to extend the time-limit before dropping the connection, e.g.:

Using the **Health IT API** from the US government, we can obtain the **Electronic Prescribing Adoption and Use by County** (see docs [here](#))

The problem is that it usually takes longer to get the data, so we pass the config option `connecttimeout` (which corresponds to the flag `--connect-timeout`) in the curl call (see next slide)

Detour on CURL options (cont. 1)

```
1 ans <- httr::GET(  
2   url      = "https://dashboard.healthit.gov/api/open-api.php",  
3   query    = list(  
4     source = "AHA_2008-2015.csv",  
5     region = "California",  
6     period = 2015  
7   ),  
8   config = config(  
9     connecttimeout = 60  
10  )  
11 )
```

```
1 > ans$request  
2 # <request>  
3 # GET https://dashboard.healthit.gov/api/open-api.php?source=AHA_2008-2015.csv&region=California&per  
4 # Output: write_memory  
5 # Options:  
6 # * useragent: libcurl/7.58.0 r-curl/4.3 httr/1.4.1  
7 # * connecttimeout: 60  
8 # * httpget: TRUE  
9 # Headers:  
10 # * Accept: application/json, text/xml, application/xml, */*
```

Regular Expressions: Email validation

This is the official regex for email validation implemented by [RFC 5322](#)

```
(?:[a-z0-9!#$%&'*/=?^_`{|}~-]+(?:\.(?:[a-z0-9!#$%&'*/=?^_`{|}~-]+)*|"(?:[\x01-\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])*")@(?:([a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.|(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?|\[(?:(?:(2(5[0-5]| [0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9]))\\.){3}(?: (2(5[0-5]| [0-4][0-9])|1[0-9][0-9]|[1-9]?[0-9])|[a-z0-9-]*[a-z0-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f]))+))\])
```

See the corresponding post in [StackOverflow](#)